

Reporte

September 27, 2026

1 Laboratorio 7

1.1 Integrantes

Nombre	Carnet	Usuario Git
Edwin Jose Gabriel De Leon Garcia	22809	EJGDLG
Gustavo Adolfo Cruz Bardales	22779	G2309
Josué Emanuel Say Garcia	22801	JosueSay
Mathew Alexander Cordero Aquino	22982	donmatthiuz

1.2 Repositorio

[Link al Repositorio](#)

1.3 Caso

Una empresa de ciberseguridad opera un sistema de detección de intrusiones en redes corporativas. El sistema monitorea el tráfico de red en tiempo real y debe decidir qué acción tomar ante comportamientos sospechosos: ignorar, alertar, bloquear temporalmente, o aislar el segmento afectado. El estado del sistema en cada instante es un vector de alta dimensión que incluye métricas de tráfico, patrones de comportamiento de usuarios, y logs de eventos recientes. La empresa quiere explorar si Deep RL puede mejorar la toma de decisiones frente a los sistemas basados en reglas que usan actualmente.

1.4 Task 1

1.4.1 Task 1.1

El equipo de ingeniería propone usar DQN estándar para este sistema. Antes de implementar cualquier cosa, usted debe hacer un análisis crítico de esa propuesta.

a. El estado del sistema es un vector de 256 dimensiones con valores continuos. Argumenten si DQN es arquitecturalmente apropiado para este tipo de estado. ¿Qué tipo de red usarían como aproximador de $Q(s, a; w)$ y por qué, considerando que el estado no es una imagen sino un vector numérico?

Respuesta: DQN es arquitecturalmente apropiado porque reemplaza la tabla Q de Q-learning por una red neuronal que aproxima $Q(s, a; w)$. Una tabla Q no es viable: un estado con 256 variables continuas produce un espacio de estados demasiado grande e imposible de enumerar. Se usaría una

red feedforward o MLP totalmente conectada, con 256 entradas y cuatro salidas, una por acción. Esta arquitectura permite aprender relaciones entre las métricas, patrones de usuarios y logs del vector. No se requiere una CNN porque el estado no tiene estructura espacial como una imagen. Tampoco se requiere una red recurrente si el vector incorpora suficiente historial reciente para representar aproximadamente el estado de Markov.

b. El espacio de acciones tiene 4 opciones discretas. Argumenten si DQN o alguna de sus variantes es apropiado para este espacio de acciones, o si es necesario un algoritmo diferente.

Respuesta: El espacio de acciones es discreto y finito: ignorar, alertar, bloquear temporalmente y aislar el segmento. Por ello DQN es apropiado, ya que puede producir un valor Q por cada acción y el máximo del target se calcula sobre solo cuatro opciones. No es necesario usar algoritmos para control continuo, como DDPG o SAC. Sin embargo, Double DQN sería una mejora recomendable frente a DQN estándar, porque reduce la sobreestimación al separar la selección de la mejor acción con la red principal y su evaluación con la red objetivo.

c. En el dominio de detección de intrusiones, los eventos críticos (intrusiones reales) son extremadamente raros: ocurren en menos del 0.1% de los pasos de tiempo. Argumenten formalmente qué consecuencia tiene esa rareza sobre el buffer de Experience Replay y sobre la distribución de muestreo uniforme. ¿Qué variante de DQN resolvería este problema específicamente y cómo?

Respuesta: Sea $p = 0.001$ la proporción máxima de intrusiones reales en el buffer. Bajo muestreo uniforme, una transición crítica tiene probabilidad p de ser seleccionada y un minibatch de tamaño B contiene en promedio Bp intrusiones. Por ejemplo, para $B = 32$, el valor esperado es $32(0.001) = 0.032$ intrusiones por minibatch y la probabilidad de no incluir ninguna es aproximadamente $(1 - 0.001)^{32} \approx 96.8\%$. Por tanto, el Experience Replay queda dominado por tráfico normal y el agente recibe muy poca señal de aprendizaje sobre eventos críticos. Prioritized Experience Replay mitiga este problema al muestrear con mayor probabilidad las transiciones de mayor error TD: $P(i) \propto (|\delta_i| + \epsilon)^\alpha$. Las intrusiones, especialmente al inicio, suelen generar errores TD altos por ser inesperadas, por lo que se reutilizan más frecuentemente que bajo muestreo uniforme.

d. El sistema actual basado en reglas tiene una tasa de falsos positivos del 8%. Si diseñaran la función de recompensa de DQN para minimizar falsos positivos únicamente, ¿qué comportamiento indeseable podría aprender el agente? Diseñen una función de recompensa con al menos tres componentes que capturen el objetivo real del sistema, justificando la magnitud relativa de cada uno.

Respuesta:

Si la recompensa minimizara únicamente los falsos positivos, el agente podría aprender a ignorar la mayoría de los comportamientos sospechosos. Esto reduciría alertas incorrectas, pero incrementaría los falsos negativos al dejar intrusiones reales sin detectar ni contener. Una función de recompensa debe balancear detección, contención y continuidad operativa:

$$r = 40I_{\text{intrusión detectada}} + 80I_{\text{intrusión contenida}} - 200I_{\text{intrusión ignorada}} - 20I_{\text{acción defensiva sobre tráfico legítimo}} - c(a)$$

$$c(\text{ignorar}) = 0, \quad c(\text{alertar}) = 1, \quad c(\text{bloquear}) = 5, \quad c(\text{aislar}) = 15$$

La penalización por ignorar una intrusión es la mayor porque un falso negativo puede causar daño grave. Detectarla tiene una recompensa positiva, y contenerla tiene una recompensa adicional para que el agente no se limite a alertar cuando bloquear o aislar sea necesario. Los falsos positivos se penalizan para mejorar la tasa actual del 8%, mientras que el costo creciente de las acciones evita bloqueos o aislamientos innecesarios.

1.4.2 Task 1.2

El equipo propone usar Double DQN en lugar de DQN estándar.

a. Expliquen formalmente el problema de sobreestimación que Double DQN resuelve. En el contexto específico de detección de intrusiones, ¿qué consecuencia operacional tendría que el agente sobreestime sistemáticamente el valor de la acción “bloquear”? ¿Y de la acción “ignorar”?

Respuesta:

DQN estándar puede sobreestimar los valores Q porque utiliza la misma red objetivo w^- para seleccionar la acción con mayor valor y para evaluar ese valor en el target. Si las estimaciones contienen ruido, el operador max tiende a escoger acciones cuyo error es positivo, produciendo una sobreestimación sistemática. Double DQN reduce este sesgo al separar la selección y la evaluación de la acción. En detección de intrusiones, sobreestimar el valor de bloquear puede llevar al agente a bloquear tráfico legítimo más de lo necesario, causando interrupciones operativas y falsos positivos. Sobreestimar el valor de ignorar puede hacer que el agente considere inofensivos eventos que requieren respuesta, aumentando falsos negativos y permitiendo que intrusiones reales continúen sin contención.

b. Escriban la expresión del target de DQN estándar y la del target de Double DQN, identificando con precisión qué parámetros hacen qué en cada caso. ¿Cuál es el único cambio de implementación entre ambos?

Respuesta:

El target de DQN estándar es:

$$y_{\text{DQN}} = r + \gamma \max_{a'} Q(s', a'; w^-)$$

Aquí, la red objetivo con parámetros w^- selecciona la acción con mayor valor y también evalúa esa misma acción.

El target de Double DQN es:

$$y_{\text{Double DQN}} = r + \gamma Q\left(s', \arg \max_{a'} Q(s', a'; w); w^-\right)$$

En Double DQN, la red principal con parámetros w selecciona la acción mediante $\arg \max$, mientras que la red objetivo con parámetros w^- evalúa el valor de la acción seleccionada. El único cambio de implementación consiste en usar la red principal para seleccionar la acción del target; la red objetivo sigue congelada y se actualiza periódicamente como en DQN estándar.

c. Argumenten si en este dominio específico la sobreestimación es un problema más grave en ciertos tipos de estados que en otros. Consideren estados con alta ambigüedad (tráfico borderline) versus estados claramente maliciosos.

Respuesta:

La sobreestimación es más grave en estados ambiguos, como tráfico borderline, porque las señales disponibles no distinguen claramente entre actividad legítima y maliciosa. En esos estados, las estimaciones Q tienen mayor incertidumbre y ruido; al aplicar el operador max, DQN tiene mayor probabilidad de elegir una acción cuyo valor está inflado por error. Por ejemplo, podría sobreestimar bloquear aunque la evidencia sea insuficiente. En estados claramente maliciosos, las diferencias entre las acciones correctas e incorrectas deberían ser más marcadas, por lo que el ruido relativo y el riesgo de seleccionar un valor inflado son menores. Por ello, Double DQN es especialmente útil para evitar decisiones excesivas ante eventos ambiguos.

1.5 Task 2

1.5.1 Task 2.1

Implementen DQN y Double DQN sobre el entorno LunarLander-v2 de Gymnasium, que tiene espacio de acción discreto con 4 acciones y estado continuo de 8 dimensiones. Usen este entorno como proxy del sistema de detección de intrusiones para validar las propiedades algorítmicas antes de escalar al dominio real.

La red para $Q(s, a; w)$ debe tener dos capas ocultas de 128 neuronas con activación ReLU. El buffer de Experience Replay debe tener capacidad de 50,000 transiciones. El minibatch debe ser de 64 transiciones. La Target Network debe actualizarse cada 500 pasos. Usen ϵ decreciente de 1.0 a 0.05 durante los primeros 10,000 pasos.

Ambos algoritmos deben implementarse desde cero usando PyTorch. No se permite usar implementaciones preconstruidas de DQN de ninguna librería de RL.

La implementación debe registrar por episodio: recompensa total, valor Q promedio estimado sobre un conjunto fijo de 100 estados de evaluación, y el target promedio usado en las actualizaciones. Estos tres registros son necesarios para detectar sobreestimación.

1.6 Task 2.1

LunarLander-v2 ya no existe en la versión actual de Gymnasium. Se usa LunarLander-v3, que es el mismo entorno con 8 dimensiones de estado y 4 acciones.

Los dos algoritmos comparten todo el código. La única diferencia está en cómo se calcula el target: DQN elige y evalúa la acción siguiente con la red target, Double DQN la elige con la red online y la evalúa con la red target.

Los tres registros por episodio se calculan así:

Registro	Cálculo
Recompensa total	Suma de recompensas del episodio
Q promedio	Promedio de $\max_a Q(s, a; w)$ con la red online sobre 100 estados fijos, medido al cerrar cada episodio
Target promedio	Promedio de y sobre todos los minibatches usados para actualizar durante el episodio

Los 100 estados de evaluación se recolectan una sola vez con una política aleatoria y semilla fija, así ambos algoritmos se miden sobre exactamente los mismos estados.

Hiperparámetros que la tarea no fija: $\gamma = 0.99$, tasa de aprendizaje de 5×10^{-4} con Adam, pérdida Huber, recorte de gradiente en norma 10 y 1,000 transiciones en el buffer antes de empezar a actualizar. Cada algoritmo entrena 600 episodios con la semilla 42.

```
[1]: import random

import numpy as np
import torch
import torch.nn as nn
import gymnasium as gym
import pandas as pd

pd.set_option("display.width", 200)

ENV_ID = "LunarLander-v3"
SEED = 42
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

HIDDEN_SIZE = 128
BUFFER_CAPACITY = 50_000
BATCH_SIZE = 64
TARGET_UPDATE_STEPS = 500
EPS_START, EPS_END, EPS_DECAY_STEPS = 1.0, 0.05, 10_000
GAMMA = 0.99
LR = 5e-4
LEARNING_STARTS = 1_000
NUM_EPISODES = 600
NUM_EVAL_STATES = 100

def set_global_seed(seed):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)

[2]: class QNetwork(nn.Module):
    def __init__(self, obs_dim, n_actions):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(obs_dim, HIDDEN_SIZE),
            nn.ReLU(),
            nn.Linear(HIDDEN_SIZE, HIDDEN_SIZE),
            nn.ReLU(),
            nn.Linear(HIDDEN_SIZE, n_actions),
        )
```

```

def forward(self, state):
    return self.net(state)

class ReplayBuffer:
    def __init__(self, capacity, obs_dim):
        self.capacity = capacity
        self.states = np.zeros((capacity, obs_dim), dtype=np.float32)
        self.actions = np.zeros(capacity, dtype=np.int64)
        self.rewards = np.zeros(capacity, dtype=np.float32)
        self.next_states = np.zeros((capacity, obs_dim), dtype=np.float32)
        self.dones = np.zeros(capacity, dtype=np.float32)
        self.pos = 0
        self.size = 0

    def add(self, state, action, reward, next_state, done):
        self.states[self.pos] = state
        self.actions[self.pos] = action
        self.rewards[self.pos] = reward
        self.next_states[self.pos] = next_state
        self.dones[self.pos] = done
        # buffer circular: al llenarse se sobrescribe la transición más vieja
        self.pos = (self.pos + 1) % self.capacity
        self.size = min(self.size + 1, self.capacity)

    def sample(self, batch_size):
        idx = np.random.randint(0, self.size, size=batch_size)
        return (
            torch.as_tensor(self.states[idx], device=DEVICE),
            torch.as_tensor(self.actions[idx], device=DEVICE),
            torch.as_tensor(self.rewards[idx], device=DEVICE),
            torch.as_tensor(self.next_states[idx], device=DEVICE),
            torch.as_tensor(self.dones[idx], device=DEVICE),
        )

```

```

[3]: def epsilon_at(step):
    frac = min(step / EPS_DECAY_STEPS, 1.0)
    return EPS_START + frac * (EPS_END - EPS_START)

def collect_eval_states(n_states, seed):
    env = gym.make(ENV_ID)
    rng = np.random.default_rng(seed)
    states = []
    obs, _ = env.reset(seed=seed)
    while len(states) < 10 * n_states:

```

```

        states.append(obs)
        obs, _, terminated, truncated, _ = env.step(int(rng.integers(env.
↪action_space.n)))
        if terminated or truncated:
            obs, _ = env.reset()
    env.close()
    idx = rng.choice(len(states), size=n_states, replace=False)
    return torch.as_tensor(np.array(states)[idx], dtype=torch.float32, ↪
↪device=DEVICE)

def compute_targets(online_net, target_net, rewards, next_states, done, ↪
↪double):
    with torch.no_grad():
        if double:
            # Double DQN: la red online elige a*, la red target la evalúa
            best_actions = online_net(next_states).argmax(dim=1, keepdim=True)
            next_q = target_net(next_states).gather(1, best_actions).squeeze(1)
        else:
            # DQN: la red target elige y evalúa con el mismo max
            next_q = target_net(next_states).max(dim=1).values
    return rewards + GAMMA * (1.0 - done) * next_q

```

```

[4]: def train_dqn(double, eval_states, num_episodes=NUM_EPISODES, seed=SEED):
    set_global_seed(seed)
    env = gym.make(ENV_ID)
    env.action_space.seed(seed)
    obs_dim = env.observation_space.shape[0]
    n_actions = env.action_space.n

    online_net = QNetwork(obs_dim, n_actions).to(DEVICE)
    target_net = QNetwork(obs_dim, n_actions).to(DEVICE)
    target_net.load_state_dict(online_net.state_dict())
    optimizer = torch.optim.Adam(online_net.parameters(), lr=LR)
    buffer = ReplayBuffer(BUFFER_CAPACITY, obs_dim)

    history = {"reward": [], "q_eval": [], "target": []}
    global_step = 0
    obs, _ = env.reset(seed=seed)
    name = "Double DQN" if double else "DQN"

    for episode in range(num_episodes):
        if episode > 0:
            obs, _ = env.reset()
            episode_reward = 0.0
            episode_targets = []
            done = False

```

```

while not done:
    if random.random() < epsilon_at(global_step):
        action = env.action_space.sample()
    else:
        with torch.no_grad():
            state_t = torch.as_tensor(obs, dtype=torch.float32,
↪device=DEVICE)
            action = int(online_net(state_t).argmax().item())

    next_obs, reward, terminated, truncated, _ = env.step(action)
    # solo terminated corta el bootstrap, truncated es un corte por
↪límite de tiempo
    buffer.add(obs, action, reward, next_obs, float(terminated))
    obs = next_obs
    episode_reward += reward
    done = terminated or truncated
    global_step += 1

    if buffer.size >= LEARNING_STARTS:
        states, actions, rewards, next_states, dones = buffer.
↪sample(BATCH_SIZE)
        targets = compute_targets(online_net, target_net, rewards,
↪next_states, dones, double)
        q_taken = online_net(states).gather(1, actions.unsqueeze(1)).
↪squeeze(1)
        loss = nn.functional.smooth_l1_loss(q_taken, targets)
        optimizer.zero_grad()
        loss.backward()
        nn.utils.clip_grad_norm_(online_net.parameters(), 10.0)
        optimizer.step()
        episode_targets.append(targets.mean().item())

    if global_step % TARGET_UPDATE_STEPS == 0:
        target_net.load_state_dict(online_net.state_dict())

    with torch.no_grad():
        q_eval = online_net(eval_states).max(dim=1).values.mean().item()

    history["reward"].append(episode_reward)
    history["q_eval"].append(q_eval)
    history["target"].append(float(np.mean(episode_targets)) if
↪episode_targets else np.nan)

    if (episode + 1) % 50 == 0:
        print(

```

```

        f"{name} episodio {episode + 1}/{num_episodes} pasos_
↪{global_step} "
        f"recompensa (50) {np.mean(history['reward'][-50:]):7.1f} "
        f"Q {q_eval:6.2f} target {history['target'][-1]:6.2f}"
    )

    env.close()
    return online_net, {k: np.array(v) for k, v in history.items()}

```

```
[5]: eval_states = collect_eval_states(NUM_EVAL_STATES, SEED)
```

```

dqn_net, dqn_history = train_dqn(double=False, eval_states=eval_states)
ddqn_net, ddqn_history = train_dqn(double=True, eval_states=eval_states)

```

```

DQN episodio 50/600 pasos 9499 recompensa (50) -135.0 Q 7.32 target
7.06

DQN episodio 100/600 pasos 58419 recompensa (50) -25.4 Q -28.34 target
9.85

DQN episodio 150/600 pasos 103156 recompensa (50) 19.1 Q 29.01 target
4.90

DQN episodio 200/600 pasos 132008 recompensa (50) 215.7 Q 58.14 target
12.16

DQN episodio 250/600 pasos 152982 recompensa (50) 233.8 Q 64.77 target
18.69

DQN episodio 300/600 pasos 167689 recompensa (50) 195.5 Q 78.39 target
25.46

DQN episodio 350/600 pasos 187399 recompensa (50) 232.9 Q 76.83 target
27.35

DQN episodio 400/600 pasos 203984 recompensa (50) 245.6 Q 88.25 target
29.31

DQN episodio 450/600 pasos 215431 recompensa (50) 164.9 Q 90.63 target
34.15

DQN episodio 500/600 pasos 226117 recompensa (50) 28.8 Q 93.35 target
41.62

DQN episodio 550/600 pasos 239105 recompensa (50) 157.8 Q 107.65 target
44.98

DQN episodio 600/600 pasos 257033 recompensa (50) 132.6 Q 99.76 target
41.45

Double DQN episodio 50/600 pasos 6137 recompensa (50) -122.5 Q 3.51
target 7.12

```

Double DQN	episodio 100/600	pasos 50518	recompensa (50)	-67.0	Q	-8.91
target	14.57					
Double DQN	episodio 150/600	pasos 96297	recompensa (50)	29.8	Q	70.50
target	5.40					
Double DQN	episodio 200/600	pasos 115934	recompensa (50)	207.8	Q	71.36
target	12.91					
Double DQN	episodio 250/600	pasos 134428	recompensa (50)	223.7	Q	66.14
target	21.72					
Double DQN	episodio 300/600	pasos 155053	recompensa (50)	224.8	Q	69.12
target	25.77					
Double DQN	episodio 350/600	pasos 177758	recompensa (50)	243.3	Q	73.54
target	21.48					
Double DQN	episodio 400/600	pasos 194558	recompensa (50)	250.1	Q	89.57
target	22.94					
Double DQN	episodio 450/600	pasos 209673	recompensa (50)	244.8	Q	86.92
target	24.84					
Double DQN	episodio 500/600	pasos 225595	recompensa (50)	269.3	Q	78.33
target	28.94					
Double DQN	episodio 550/600	pasos 241907	recompensa (50)	245.9	Q	82.17
target	29.13					
Double DQN	episodio 600/600	pasos 256879	recompensa (50)	248.1	Q	83.08
target	31.20					

```
[6]: def summarize(history, block=100):
    rows = []
    for start in range(0, len(history["reward"]), block):
        end = start + block
        rows.append({
            "episodios": f"{start + 1}-{end}",
            "recompensa": np.mean(history["reward"][start:end]),
            "Q promedio": np.nanmean(history["q_eval"][start:end]),
            "target promedio": np.nanmean(history["target"][start:end]),
        })
    return pd.DataFrame(rows).set_index("episodios").round(2)

pd.concat({"DQN": summarize(dqn_history), "Double DQN":
    ↪summarize(ddqn_history)}, axis=1)
```

	DQN				Double DQN				
	recompensa	Q	promedio	target	promedio	recompensa	Q	promedio	target
promedio									

episodios					
1-100	-80.19	-8.12	6.17	-94.75	2.13
6.86					
101-200	117.42	20.68	6.54	118.79	56.09
7.82					
201-300	214.64	63.13	19.09	224.27	67.30
20.96					
301-400	239.23	82.05	27.61	246.70	76.28
22.57					
401-500	96.85	91.45	34.54	257.05	84.31
24.94					
501-600	145.17	103.38	43.21	246.98	80.32
29.63					

1.6.1 Task 2.2

Con base en los resultados de la implementación, respondan:

- Grafiquen la recompensa promedio por episodio de DQN y Double DQN en la misma figura con media móvil de 20 episodios. ¿Cuál converge más rápido? ¿Cuál alcanza mayor recompensa final? ¿El resultado coincide con lo esperado teóricamente?
- Grafiquen el valor Q promedio estimado durante el entrenamiento para ambos algoritmos. El valor Q de DQN debería ser sistemáticamente mayor que el de Double DQN sobre los mismos estados. ¿Observan ese patrón? Si no lo observan, argumenten por qué podría no ser visible en este entorno específico.
- Implementen ahora Prioritized Experience Replay sobre Double DQN, usando $\alpha = 0.6$ y β creciente de 0.4 a 1.0. Comparen la velocidad de convergencia de Double DQN con muestreo uniforme versus Double DQN con PER. ¿En qué fase del entrenamiento es más visible la diferencia?
- Argumenten si PER sería más o menos beneficioso en el dominio real de detección de intrusiones comparado con LunarLander, considerando la rareza de los eventos críticos que analizaron en la Task 1.1c. Apoyen su argumento con evidencia de sus experimentos.

1.6.2 Task 2.3

Investigación bibliográfica y dictamen técnico.

- Busquen y lean un paper publicado entre 2022 y 2025 que aplique DQN, Double DQN, Dueling DQN, PER, o Rainbow a un problema de ciberseguridad, detección de anomalías, o gestión de redes. El paper debe ser de IEEE Transactions on Information Forensics and Security, NeuroIPS, ICML, USENIX Security, o similar. Escriban un resumen técnico de media página con: el problema que resuelve, qué variante de DQN usa y por qué, los resultados principales, y una reflexión sobre si las limitaciones de DQN que identificaron en la Task 1 fueron resueltas en ese trabajo.
- Redacten un dictamen técnico de un párrafo dirigido al equipo directivo de la empresa de ciberseguridad. El dictamen debe argumentar si recomiendan proceder con DQN o alguna de sus variantes para el sistema de detección de intrusiones, qué variante específica recomiendan y por qué, y qué pasos adicionales serían necesarios antes de un despliegue en producción. El dictamen debe estar respaldado por evidencia de sus experimentos, no solo por argumentos teóricos.